

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра информационных систем

КУРСОВАЯ РАБОТА

по дисциплине «Методы и средства проектирования информационных
систем»

Тема: «Разработка приложения для работы со справочником»

Студент гр. 3376	_____	Михайлов Н.Ф.
Студентка гр. 3376	_____	Дегтярева М. И.
Студент гр. 3376	_____	Константинов Р. И.
Преподаватель	_____	Дюков Н.В.

Санкт-Петербург
2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студент Михайлов Н.Ф.

Студентка Дегтярева М. И.

Студент Константинов Р. И.

Группа 3376

Тема работы: Разработка приложения для работы со справочником

Исходные данные:

Создать приложение для работы со справочником изделий, которое хранит классификатор, товары, параметры, перечисления и единицы измерения в базе данных, предоставляет доступ к данным посредством API сервера и содержит пользовательский web-интерфейс для просмотра, фильтрации и управления данными справочника.

Содержание пояснительной записки:

«Аннотация», «Содержание», «Введение», «Анализ предметной области», «Разработка объектной модели этапа проектирования», «Разработка модели хранения», «Разработка базы данных», «Разработка основных процедур», «Разработка пользовательского интерфейса», «Результаты тестирования», «Заключение», «Список использованных источников», «Приложение».

Предполагаемый объем пояснительной записки:

Не менее 25 страниц.

Дата выдачи задания: 01.04.2026

Дата сдачи реферата: 28.05.2026

Дата защиты реферата: 28.05.2026

Студент(ка)	_____	Михайлов Н.Ф.
Студент(ка)	_____	Дегтярева М. И.
Студент(ка)	_____	Константинов Р. И.
Преподаватель	_____	Дюков Н.В.

АННОТАЦИЯ

В работе описана разработка web-приложения для справочника электронных комплектующих. Приложение хранит категории, товары, характеристики, параметры, перечисления и единицы измерения в PostgreSQL, а доступ к данным организован через API на FastAPI. Для пользователя сделана витрина с фильтрами и корзиной, для администрирования предусмотрены формы управления категориями и товарами. В пояснительной записке приведены анализ предметной области, структура базы данных, архитектура приложения, описание интерфейса и результаты проверки основных сценариев.

SUMMARY

This paper describes the development of a web application for an electronic components catalog. The application stores categories, products, specifications, parameters, enumerations and measurement units in PostgreSQL, and provides access to the data through a FastAPI-based API. The user interface includes a product showcase with filters and a shopping cart. Administrative forms are provided for managing categories and products. The explanatory note includes the domain analysis, database structure, application architecture, interface description and the results of testing the main scenarios.

Содержание

Задание на курсовую работу	1
Аннотация	2
1 Введение	4
2 Основная часть	5
2.1 Анализ предметной области	5
2.1.1 Цели и задачи	5
2.1.2 Описание пользовательской аудитории	6
2.1.3 Диаграмма Use-case	7
2.2 Разработка приложения	9
2.2.1 Архитектура информационной системы	9
2.2.2 Диаграмма классов	10
2.2.3 ERD диаграмма	11
2.2.4 Технологии разработки	13
2.2.5 Тестирование приложения	13
2.3 Разработка пользовательского интерфейса	23
2.3.1 Информационная архитектура	23
2.3.2 Диаграмма потоков задач	23
2.3.3 Выбор цветов и шрифтов	24
2.3.4 Страницы интерфейса	24
3 Заключение	26
А Приложение А. Скриншоты интерфейса	28
В Приложение Б. Фрагменты программного кода	29

1 ВВЕДЕНИЕ

В данной курсовой работе разрабатывается web-приложение для работы со справочником электронных комплектующих. Предметная область выбрана на примере каталога электроники: процессоров, видеокарт, материнских плат и других товаров.

Цель работы — спроектировать и разработать приложение, которое позволяет хранить классификатор изделий в базе данных, управлять товарами и их параметрами через API, а также работать с этими данными через пользовательский web-интерфейс.

Для достижения цели необходимо решить следующие задачи:

- проанализировать предметную область и определить основные сценарии работы пользователей;
- разработать модель категорий, товаров, характеристик, параметров, перечислений и единиц измерения;
- реализовать хранение данных в PostgreSQL;
- разработать REST API для управления справочником;
- создать web-интерфейс для просмотра каталога, фильтрации товаров и работы с корзиной;
- предусмотреть административные формы для добавления и редактирования основных сущностей;
- проверить работу приложения на основных сценариях.

Практическая значимость работы состоит в том, что пользователь получает доступ к справочнику через браузер без установки дополнительного программного обеспечения. Администратор может изменять структуру каталога и наполнение справочника, а пользователь витрины — искать товары по категории, цене, наличию и характеристикам.

2 ОСНОВНАЯ ЧАСТЬ

2.1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1.1 ЦЕЛИ И ЗАДАЧИ

Цель ИС – автоматизация ведения справочника электронных комплектующих и предоставление удобного доступа к данным через web-интерфейс и API. Система должна хранить классификатор изделий, товары, характеристики, параметры, перечисления и единицы измерения в базе данных. Пользователь работает со справочником через витрину, а администратор управляет структурой и наполнением каталога.

Требования к контенту и функционалу:

Администратор может:

- создавать, редактировать и удалять категории справочника;
- добавлять товары электронных комплектующих и изменять их основные данные;
- задавать характеристики товаров, в том числе значения с единицами измерения;
- создавать перечисления и наполнять их значениями, например сокетам, форм-факторами или типами памяти;
- создавать параметры разных типов и привязывать их к категориям товаров;
- задавать ограничения для числовых параметров;
- использовать наследование параметров от родительских категорий;
- проверять работу справочника через API и административные формы интерфейса.

Пользователь может:

- просматривать каталог электронных комплектующих;
- выбирать категорию товаров;
- искать товары по названию и описанию;
- фильтровать товары по цене, наличию и характеристикам;
- просматривать краткую информацию о товаре и его параметрах;
- добавлять товары в корзину, изменять количество и удалять позиции из корзины;
- пользоваться системой без доступа к изменению справочных данных.

2.1.2 ОПИСАНИЕ ПОЛЬЗОВАТЕЛЬСКОЙ АУДИТОРИИ

В системе выделяются две основные группы пользователей:

- администратор – пользователь, который отвечает за ведение справочника. Он работает с категориями, товарами, параметрами, перечислениями и единицами измерения. Для него важны быстрые формы редактирования, понятные ошибки валидации и возможность проверить результат на витрине;
- пользователь витрины – человек, которому нужно найти подходящий товар в каталоге. Он не изменяет данные, а просматривает товары, применяет фильтры и собирает корзину. Для него важны простая навигация, минимум лишних действий и понятное отображение характеристик.

Интерфейс рассчитан на работу с компьютера и мобильного устройства. На desktop-экране основные разделы доступны через верхнее меню. На мобильной версии основные действия вынесены в нижнюю панель навигации, чтобы каталог, витрина, корзина и административный блок оставались доступны без лишней прокрутки.

2.1.3 ДИАГРАММА USE-CASE

Для справочника электронных комплектующих выделены два актора: пользователь витрины и администратор справочника.

Пользователь витрины работает с публичной частью приложения. Он просматривает каталог, выбирает категории, применяет фильтры, смотрит характеристики товаров и добавляет позиции в корзину. Этот актер не изменяет справочные данные.

Администратор справочника отвечает за наполнение и поддержку данных. Он управляет категориями, товарами, параметрами, перечислениями и единицами измерения. Через административные формы и API он может изменять структуру справочника и проверять результат на витрине.

Основные акторы системы:

- пользователь витрины;
- администратор справочника.

Основные варианты использования:

- просмотр каталога;
- фильтрация товаров;
- добавление товара в корзину;
- изменение количества товара в корзине;
- управление категориями;
- управление товарами;
- управление параметрами, перечислениями и единицами измерения.

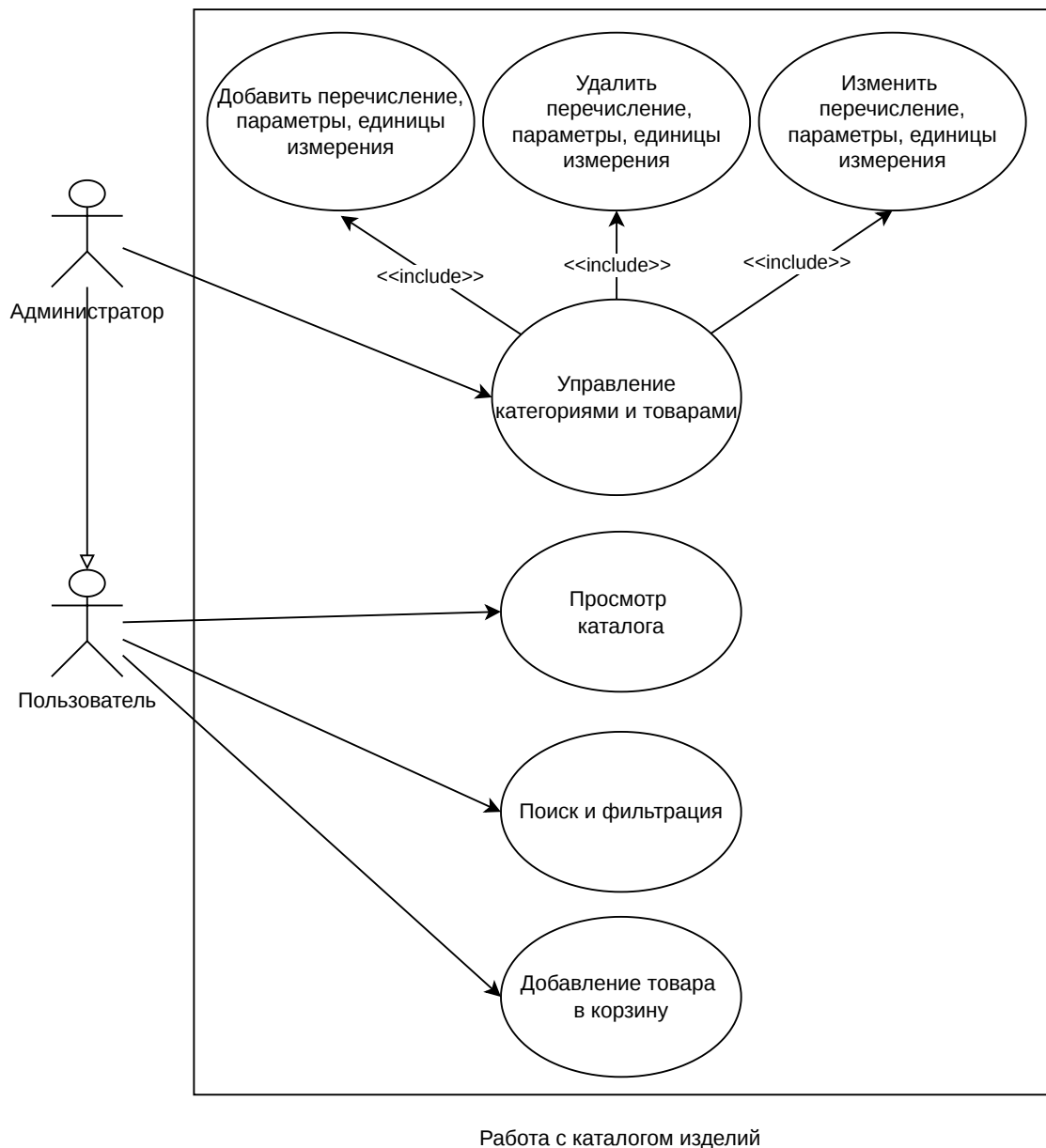


Рис. 1: Диаграмма вариантов использования

Диаграмма отражает разделение функций между обычным пользователем и администратором. Пользовательские сценарии связаны с поиском и выбором товаров, а административные сценарии – с изменением справочных данных. Такое разделение упрощает интерфейс: пользователь видит только витрину и корзину, а инструменты редактирования вынесены в отдельный административный блок.

2.2 РАЗРАБОТКА ПРИЛОЖЕНИЯ

2.2.1 АРХИТЕКТУРА ИНФОРМАЦИОННОЙ СИСТЕМЫ

Приложение представляет собой модульное монолитное web-приложение. Серверная часть, API, логика работы со справочником и отдача статических файлов находятся в одном Python-приложении. Отдельного frontend-сервера нет: страница интерфейса, стили и JavaScript-файл отдаются тем же FastAPI-приложением, которое обрабатывает REST-запросы.

Точка входа приложения находится в файле `src/main.py`. При запуске создаётся объект FastAPI, подключаются маршруты API, монтируется директория `src/static`, а корневой адрес `/` отдаёт файл `index.html`. В жизненном цикле приложения также выполняется подготовка базы данных: создаётся база при необходимости, создаются таблицы и закрывается подключение при остановке сервера.

Компоненты системы:

1. Серверное приложение на FastAPI. Оно принимает HTTP-запросы, маршрутизирует их и возвращает ответы в формате JSON. Основные маршруты собраны в `src/api/database.py`; общий роутер подключается через `src/api/__init__.py`. Для проверки работы сервера и получения общей информации используется отдельный модуль `src/api/info.py`.
2. Модели запросов и ответов. В файле `src/api/models.py` описаны Pydantic-модели для категорий, товаров, спецификаций, параметров, перечислений, единиц измерения и значений параметров. Эти модели проверяют входные данные до передачи их в слой работы с базой.
3. Слой работы с базой данных. Основная логика находится в `src/db/database.py`. В этом файле описаны таблицы SQLAlchemy Core, ограничения, связи между таблицами и функции для создания, чтения, изменения и удаления данных. Здесь же реализована проверка бизнес-правил: например, запрет отрицательной цены, проверка значений перечислений и проверка диапазонов числовых параметров.

4. База данных PostgreSQL. PostgreSQL используется как внешнее хранилище справочника. В базе находятся категории, товары, характеристики, перечисления, значения перечислений, единицы измерения, параметры категорий и значения параметров конкретных товаров.
5. Статический frontend. Пользовательский интерфейс состоит из файлов `src/static/index.html`, `src/static/styles.css` и `src/static/app.js`. Клиентская часть загружает данные через REST API, отображает витрину, фильтры, корзину и административные формы. Корзина хранится на стороне браузера в `localStorage`.
6. Контейнеризация. Для запуска проекта используется `Dockerfile` и `docker-compose`. В `compose`-файле описаны два сервиса: API-приложение и PostgreSQL. API запускается после проверки готовности базы данных.

Таким образом, приложение остаётся монолитным с точки зрения развёртывания, но внутри разделено на понятные модули: входная точка, API, схемы данных, функции работы с БД и статический интерфейс. Такой вариант проще поддерживать в рамках курсовой работы и достаточно хорошо подходит для справочника, где все основные операции связаны с одной базой данных.

2.2.2 ДИАГРАММА КЛАССОВ

Диаграмма классов отражает основные классы предметной области и интерфейсные классы web-приложения. Классы предметной области описывают структуру справочника изделий, параметры, перечисления и единицы измерения. Интерфейсные классы соответствуют основным экранам и блокам пользовательского интерфейса: витрине, фильтрам, корзине, форме входа и административным формам.

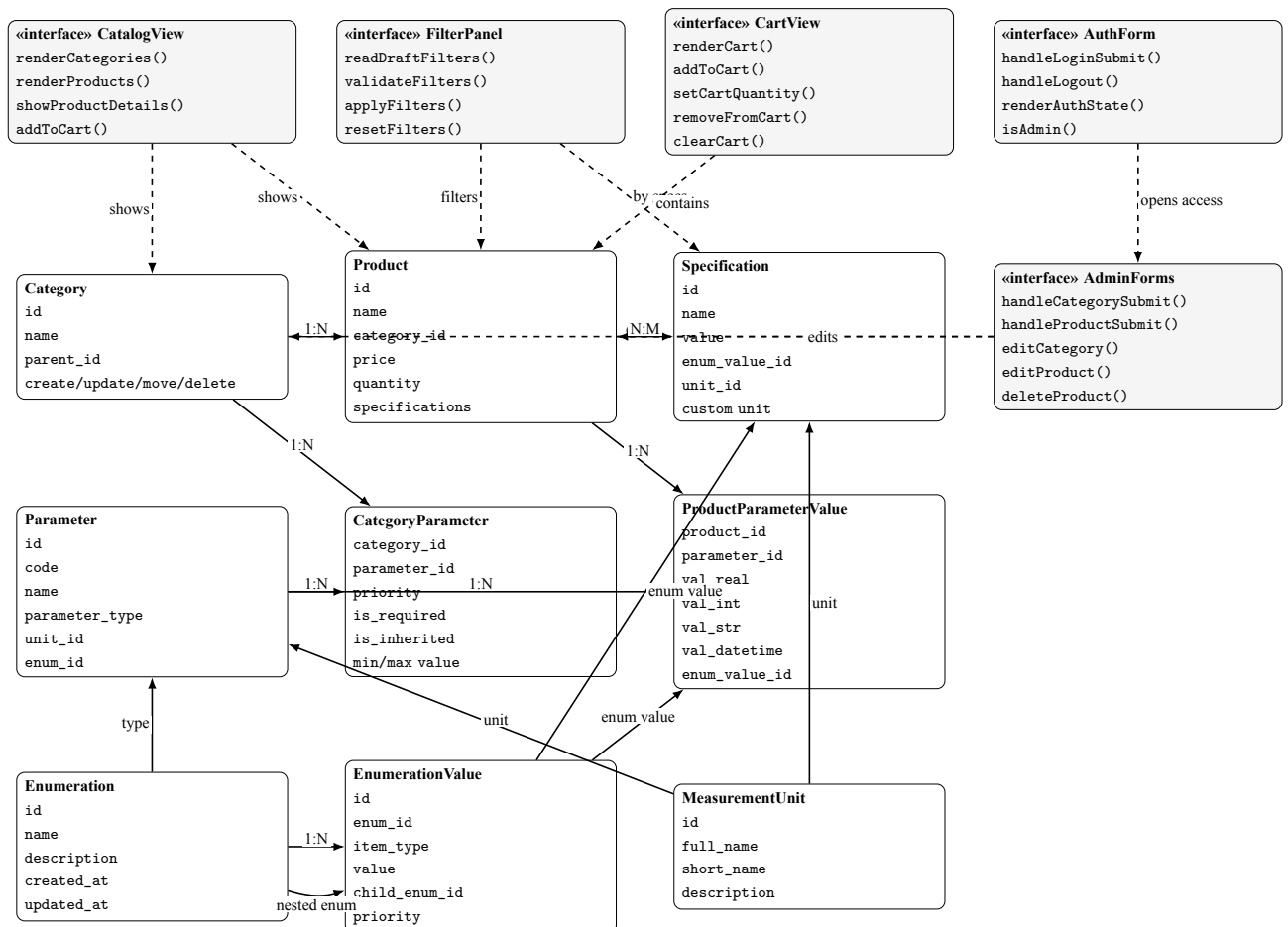


Рис. 2: Диаграмма классов приложения

На диаграмме показано, что центральным классом справочника является `Product`, который относится к категории, имеет набор характеристик и значения настраиваемых параметров. Классы `Enumeration`, `EnumerationValue` и `MeasurementUnit` используются как вспомогательные справочники для параметров и характеристик. Интерфейсные классы отделены от предметных классов пунктирными зависимостями: они не хранят данные самостоятельно, а отображают их пользователю и вызывают операции чтения или изменения через API.

2.2.3 ERD ДИАГРАММА

ERD диаграмма построена по фактической структуре таблиц, описанной в модуле `src/db/database.py`. В модели выделены сущности для дерева категорий, товаров, характеристик, параметров, перечислений, единиц измерения и таблиц связей, которые позволяют назначать параметры категориям и хранить

значения параметров конкретных товаров.

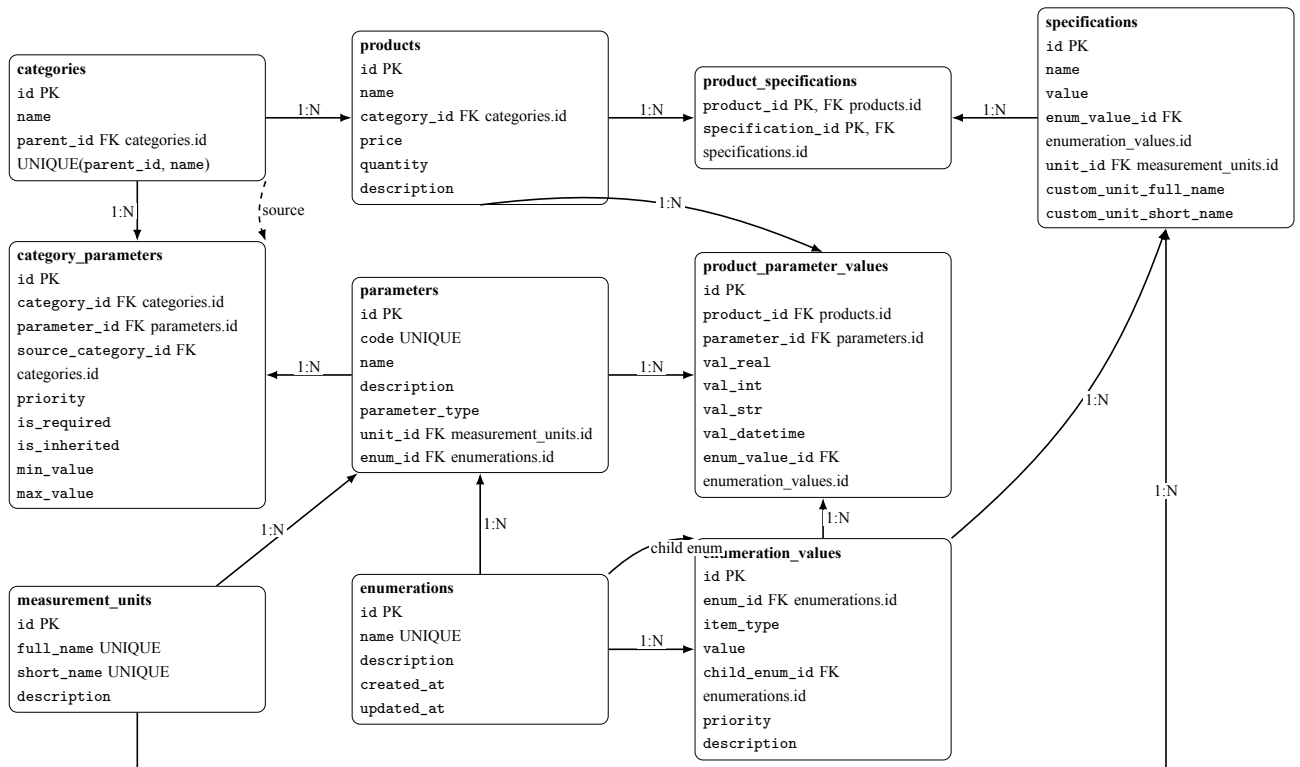


Рис. 3: ERD диаграмма базы данных приложения

На диаграмме сплошные линии показывают основные внешние ключи, а пунктирная связь используется для указания исходной категории при наследовании параметров. Таблица `enumeration_values` хранит как конечные значения перечислений, так и ссылки на вложенные перечисления через поле `child_enum_id`, поэтому она связана с таблицей `enumerations` двумя внешними ключами.

Основные таблицы проекта:

- `categories` – дерево категорий справочника;
- `products` – товары, привязанные к категориям;
- `specifications` – характеристики товаров;
- `product_specifications` – связь товаров с характеристиками;
- `measurement_units` – справочник единиц измерения;
- `enumerations` – справочник перечислений;
- `enumeration_values` – значения перечислений и вложенные перечисления;

- `parameters` – метаданные параметров;
- `category_parameters` – параметры, назначенные категориям;
- `product_parameter_values` – значения параметров конкретных товаров.

2.2.4 ТЕХНОЛОГИИ РАЗРАБОТКИ

Для разработки серверной части выбран FastAPI, так как он позволяет быстро создавать REST API, автоматически формирует Swagger-документацию и поддерживает асинхронную работу. Для хранения данных используется PostgreSQL. Работа с базой данных выполняется через SQLAlchemy Core.

Пользовательский интерфейс реализован без отдельного frontend-фреймворка: HTML, CSS и JavaScript отдаются самим FastAPI-приложением. Такой подход упрощает запуск проекта и соответствует формату курсовой работы.

Использованные технологии:

- Python;
- FastAPI;
- PostgreSQL;
- SQLAlchemy Core;
- HTML;
- CSS;
- JavaScript;
- Docker.

2.2.5 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

Функциональное тестирование проводилось вручную по основным сценариям использования web-приложения и REST API. Проверялись публичные

функции витрины, работа корзины, авторизация администратора и операции изменения справочных данных. Ниже приведены тестовые сценарии и места для скриншотов, подтверждающих выполнение проверок.

1. Авторизация и разграничение прав.

Сценарий: пользователь открывает сайт без авторизации и просматривает витрину товаров. Затем выполняется вход под учетной записью администратора через форму в административном разделе.

Ожидаемый результат: без авторизации доступны каталог, витрина, фильтры и корзина, но кнопки редактирования и формы управления скрыты. После входа администратора отображаются формы создания категорий и товаров, а на карточках появляются кнопки изменения и удаления.

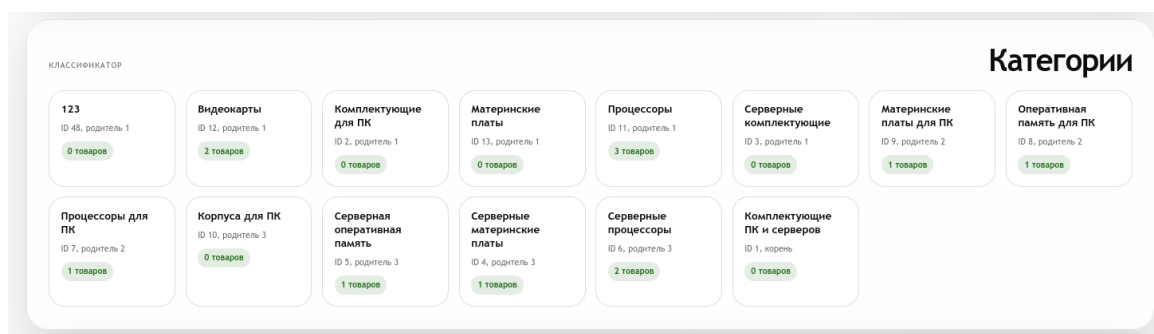


Рис. 4: Витрина в режиме просмотра

УПРАВЛЕНИЕ

Быстрое добавление

Для просмотра каталога вход не нужен. Для создания, изменения и удаления данных необходимо войти как администратор.

Вход выполнен: admin, роль администратора.

ЛОГИН

ПАРОЛЬ

Войти

Выйти

Вход выполнен. Доступно редактирование данных.

Рис. 5: Вход администратора

КЛАССИФИКАТОР

Категории

123 ID 48, родитель 1 0 товаров Изменить Удалить	Видеокарты ID 12, родитель 1 2 товаров Изменить Удалить	Комплектующие для ПК ID 2, родитель 1 0 товаров Изменить Удалить	Материнские платы ID 13, родитель 1 0 товаров Изменить Удалить	Процессоры ID 11, родитель 1 3 товаров Изменить Удалить	Серверные комплектующие ID 3, родитель 1 0 товаров Изменить Удалить	Материнские платы для ПК ID 9, родитель 2 1 товаров Изменить Удалить	Оперативная память для ПК ID 8, родитель 2 1 товаров Изменить Удалить
Процессоры для ПК ID 7, родитель 2 1 товаров Изменить Удалить	Корпуса для ПК ID 10, родитель 3 0 товаров Изменить Удалить	Серверная оперативная память ID 5, родитель 3 1 товаров Изменить Удалить	Серверные материнские платы ID 4, родитель 3 1 товаров Изменить Удалить	Серверные процессоры ID 6, родитель 3 2 товаров Изменить Удалить	Комплектующие ПК и серверов ID 1, корень 0 товаров Изменить Удалить		

Рис. 6: Отображение форм управления

2. Создание категории.

Сценарий: администратор переходит в административный блок, заполняет название новой категории, выбирает родительский раздел и нажимает кнопку создания.

Ожидаемый результат: новая категория сохраняется в базе данных, появляется в списке категорий и становится доступной в выпадающем списке фильтра по категории.

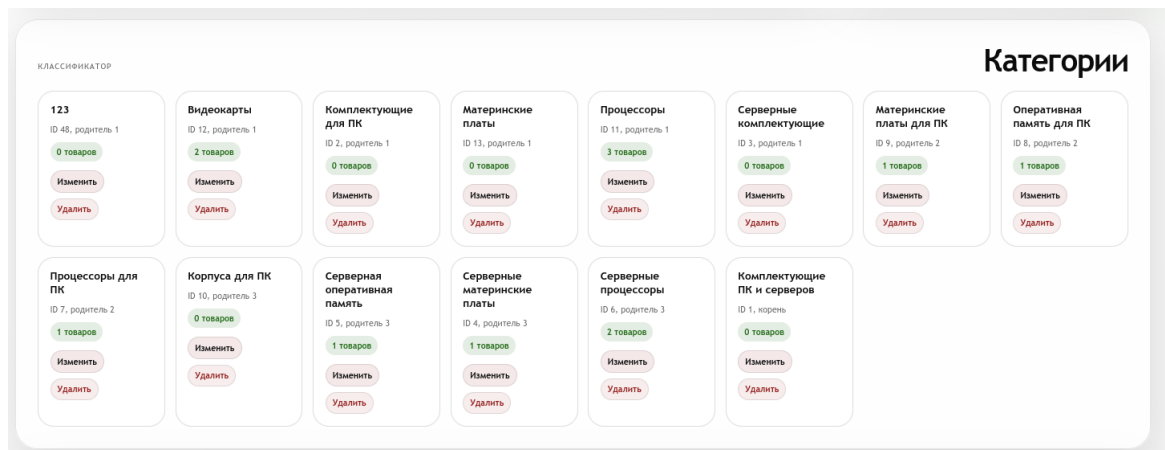


Рис. 7: Отображение форм управления до создания категории

Новая категория

НАЗВАНИЕ

Готовые Игровые ПК

РОДИТЕЛЬ

Корневой раздел

Создать категорию

Отмена

Рис. 8: Создание категории

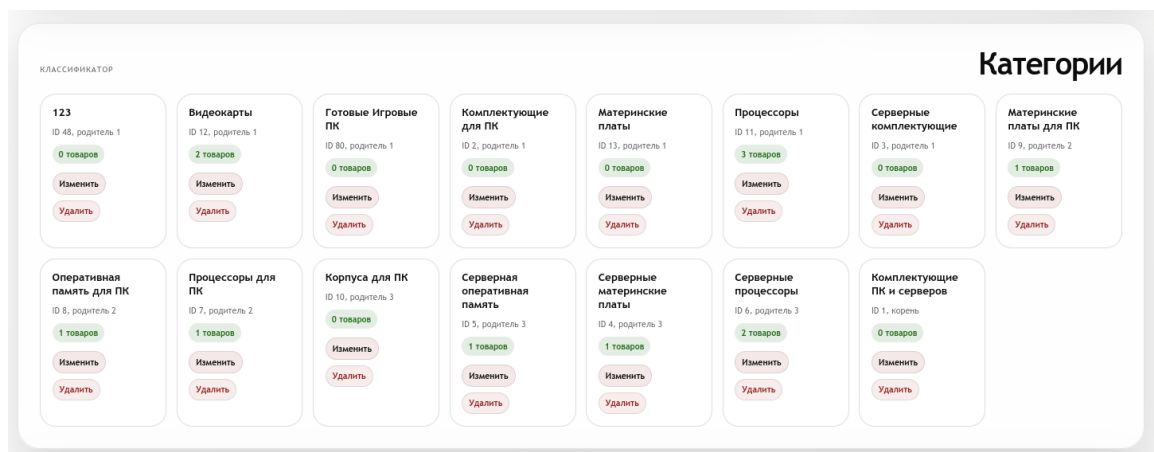


Рис. 9: Категория добавлена в классификатор

3. Создание товара с характеристиками.

Сценарий: администратор заполняет форму нового товара, указывает категорию, цену, остаток и описание. После сохранения товар добавляется в справочник и отображается на витрине.

Ожидаемый результат: товар создается через защищенный endpoint, появляется в карточках витрины, а его цена, остаток, категория и описание отображаются корректно.

Рис. 10: Создание товара

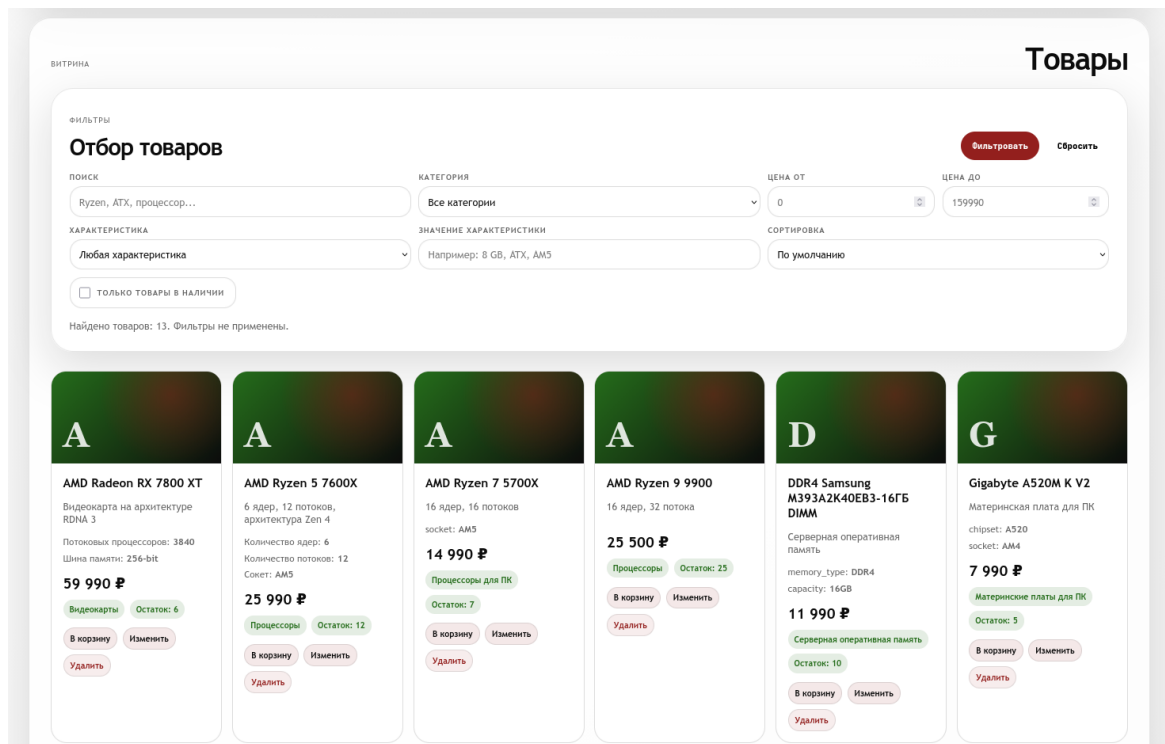


Рис. 11: Новый товар на витрине

4. Управление перечислениями.

Сценарий: администратор создает перечисление, например «Сокеты процессоров», и добавляет значения AM4, AM5 и LGA1700. Затем значение перечисления используется в характеристике товара.

Ожидаемый результат: перечисление и его значения сохраняются в таблицах `enumerations` и `enumeration_values`. На витрине характеристика отображается названием значения, например AM5, а не техническим идентификатором.

Новое перечисление

НАЗВАНИЕ

ОПИСАНИЕ

Создать перечисление

Значение перечисления

ПЕРЕЧИСЛЕНИЕ

ТИП ЭЛЕМЕНТА

ЗНАЧЕНИЕ

ВЛОЖЕННОЕ ПЕРЕЧИСЛЕНИЕ

ПРИОРИТЕТ

ОПИСАНИЕ

Добавить значение

Значения пока не добавлены.

Рис. 12: Создание перечисления и его значений

A

AMD Radeon RX 7800 XT

Видеокарта на архитектуре RDNA 3

Потоковых процессоров: 3840

Шина памяти: 256-bit

59 990 ₺

Видеокарты

Остаток: 6

В корзину

Изменить

Удалить

A

AMD Ryzen 5 7600X

6 ядер, 12 потоков, архитектура Zen 4

Количество ядер: 6

Количество потоков: 12

Сокет: AM5

25 990 ₺

Процессоры

Остаток: 12

В корзину

Изменить

Удалить

A

AMD Ryzen 7 5700X

16 ядер, 16 потоков

socket: AM5

14 990 ₺

Процессоры для ПК

Остаток: 7

В корзину

Изменить

Удалить

A

AMD Ryzen 9 9900

16 ядер, 32 потока

25 500 ₺

Процессоры

Остаток: 25

В корзину

Изменить

Удалить

D

DDR4 Samsung M393A2K40EB3-16GB DIMM

Серверная оперативная память

memory_type: DDR4

capacity: 16GB

11 990 ₺

Серверная оперативная память

Остаток: 10

В корзину

Изменить

Удалить

G

Gigabyte A520M K V2

Материнская плата для ПК

chipset: A520

socket: AM4

7 990 ₺

Материнские платы для ПК

Остаток: 5

В корзину

Изменить

Удалить

Рис. 13: Отображение значения перечисления в характеристиках товара

5. Поиск и фильтрация товаров на витрине.

Сценарий: пользователь задает фильтры на витрине: категорию, диапазон цены, наличие на складе, характеристику и значение характеристики. После нажатия кнопки «Фильтровать» выполняется проверка введенных данных.

Ожидаемый результат: при корректных значениях витрина показывает

только подходящие товары. При некорректном диапазоне, например отрицательной цене или нижней границе больше верхней, появляется всплывающее сообщение, а фильтрация не выполняется.

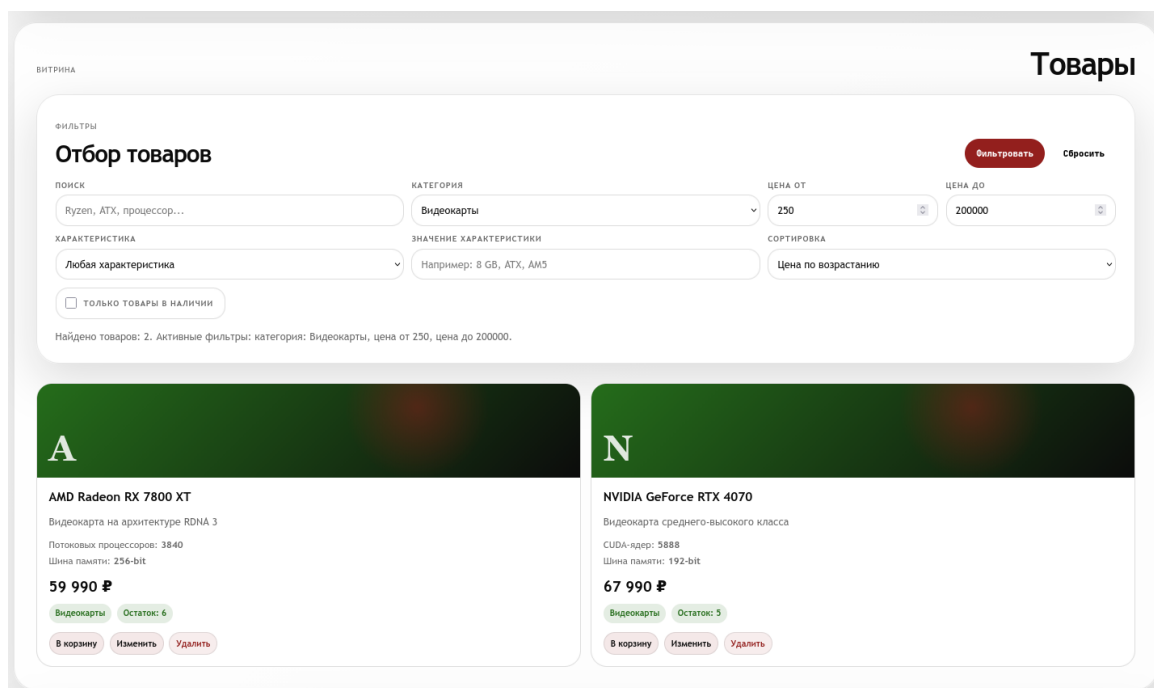


Рис. 14: Результат фильтрации товаров

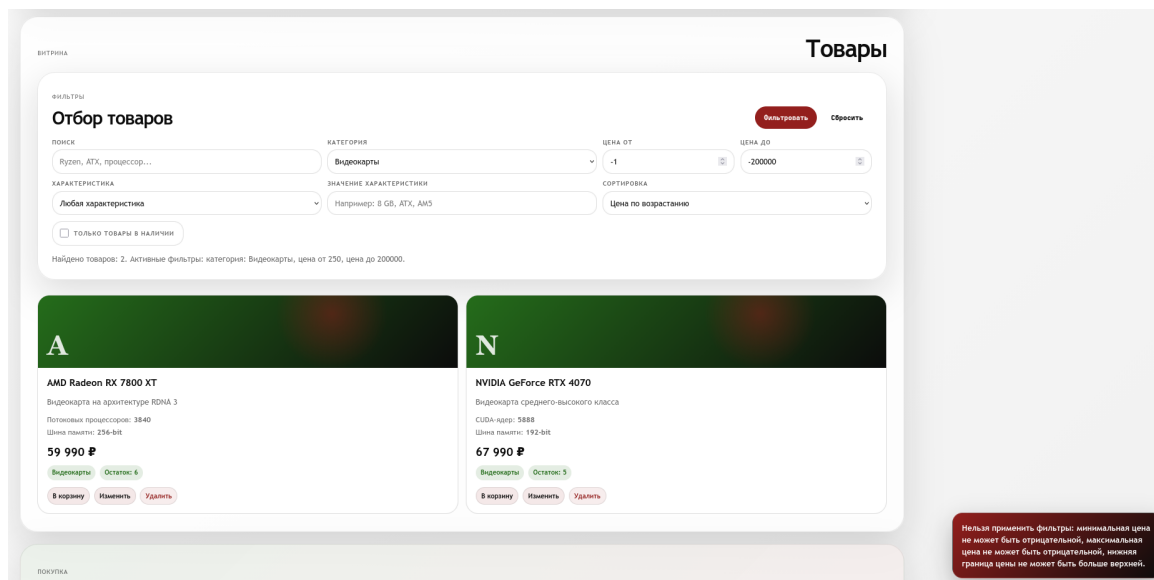


Рис. 15: Ошибка валидации фильтров

6. Работа с корзиной.

Сценарий: пользователь добавляет товар в корзину, изменяет количество, удаляет одну позицию и очищает корзину. Затем нажимает кнопку «Заказать».

Ожидаемый результат: количество товаров и итоговая сумма пересчитываются на стороне клиента. При нажатии кнопки «Заказать» выводится уведомление о том, что оформление заказа пока не реализовано.

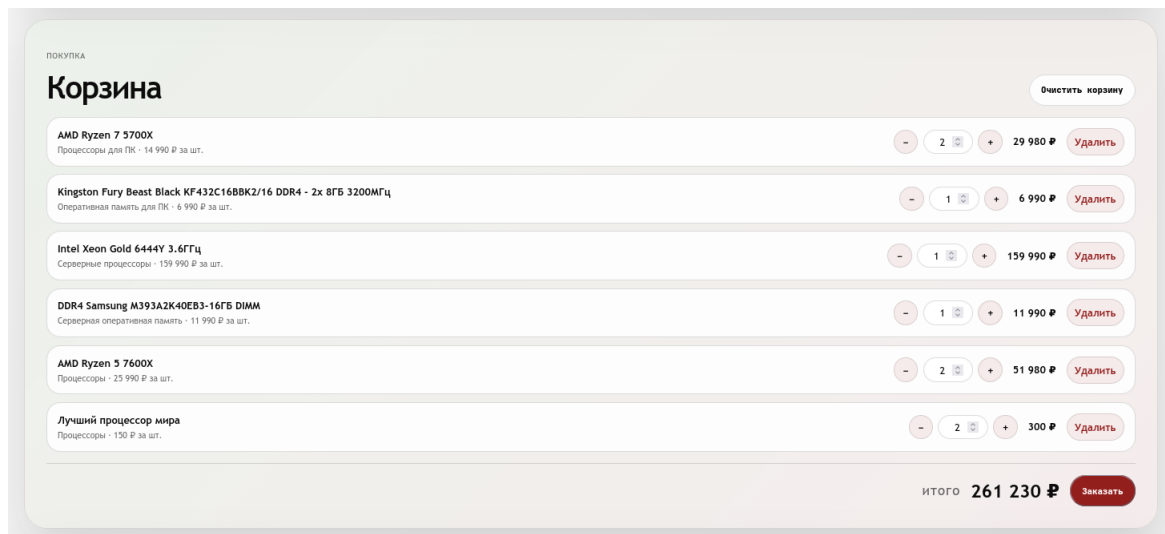


Рис. 16: Корзина с добавленными товарами

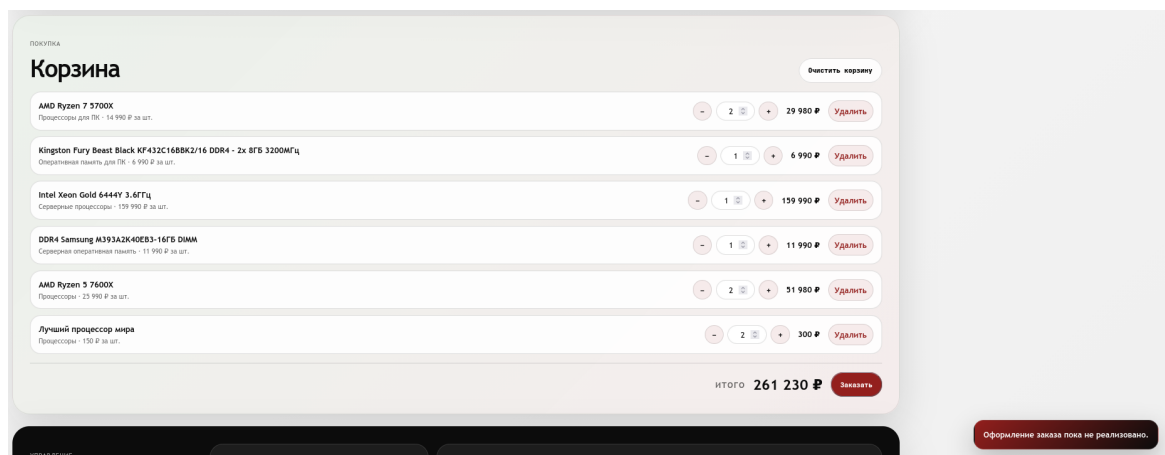


Рис. 17: Уведомление при попытке оформления заказа

Все описанные сценарии завершились успешно. Публичные операции чтения доступны пользователю без авторизации, а операции создания, редактирования и удаления выполняются только после входа администратора. Ошибки валидации отображаются пользователю в виде понятных сообщений, а серверные ограничения предотвращают запись некорректных данных в базу.

2.3 РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

2.3.1 ИНФОРМАЦИОННАЯ АРХИТЕКТУРА

Информационная архитектура web-интерфейса включает следующие основные разделы:

- главная страница;
- каталог категорий;
- витрина товаров;
- фильтры товаров;
- корзина;
- административный блок;
- Swagger API.

Навигация на desktop-устройствах расположена в верхней части страницы. Для мобильных устройств реализована нижняя панель навигации с основными действиями.

2.3.2 ДИАГРАММА ПОТОКОВ ЗАДАЧ

В данном разделе необходимо вставить Task Flow диаграмму.

Основные пользовательские потоки:

- просмотр категории → выбор фильтров → фильтрация → просмотр товара;
- просмотр товара → добавление в корзину → изменение количества → оформление заказа;
- администратор → создание категории → создание товара → проверка товара на витрине;

- администратор → создание перечисления → добавление значения → использование значения в характеристике.

2.3.3 ВЫБОР ЦВЕТОВ И ШРИФТОВ

Для интерфейса выбрана контрастная цветовая палитра:

- #931F1D – акцентный цвет для основных кнопок и уведомлений;
- #256D1B – дополнительный зелёный цвет для меток и элементов состояния;
- #0C0C0C – основной тёмный цвет текста и контрастных блоков;
- #FFFFFF – базовый цвет поверхностей и фона.

Выбор палитры связан с необходимостью обеспечить контрастность интерфейса и визуально выделить ключевые действия пользователя. Основной шрифт интерфейса – sans-serif, для крупных заголовков используется serif-шрифт, что делает главную страницу более выразительной.

2.3.4 СТРАНИЦЫ ИНТЕРФЕЙСА

В данном разделе необходимо привести скриншоты разработанных страниц и элементов интерфейса.

Реализованные элементы:

- главная страница с краткой статистикой;
- каталог категорий;
- витрина товаров;
- фильтры товаров с валидацией;
- карточка товара;
- корзина;

- административные формы добавления и редактирования категорий и товаров;
- всплывающие уведомления.

3 ЗАКЛЮЧЕНИЕ

В ходе курсовой работы было разработано web-приложение для работы со справочником электронных комплектующих. Была реализована серверная часть, база данных, API и пользовательский интерфейс. Система поддерживает работу с категориями, товарами, характеристиками, перечислениями, единицами измерения, параметрами и корзиной.

В данном разделе необходимо дополнить выводы о выполненных задачах, достигнутых результатах и возможных направлениях развития проекта.

СПИСОК ЛИТЕРАТУРЫ

- [1] FastAPI Documentation [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/>
- [2] PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/>
- [3] SQLAlchemy Documentation [Электронный ресурс]. URL: <https://docs.sqlalchemy.org/>
- [4] Web Content Accessibility Guidelines (WCAG) 2.1 [Электронный ресурс]. URL: <https://www.w3.org/TR/WCAG21/>
- [5] ГОСТ Р ИСО 9241-161-2016. Эргономика взаимодействия человек-система. Элементы графического пользовательского интерфейса.

А ПРИЛОЖЕНИЕ А. СКРИНШОТЫ ИНТЕРФЕЙСА

В приложение необходимо добавить скриншоты главной страницы, витрины, фильтров, корзины и административных форм.

В ПРИЛОЖЕНИЕ Б. ФРАГМЕНТЫ ПРОГРАММНОГО КОДА

В приложение при необходимости можно добавить фрагменты SQL-процедур, API-обработчиков и JavaScript-кода пользовательского интерфейса.